Content-Aware Image Compression via LLM-Guided Semantic Segmentation

Sujoy Datta
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
sdattafcs@kiit.ac.in

Mobasshir Ajaz
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
mobasshirajaz@gmail.com

Daithoulung Rongmai
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
daithoulungrongmai@gmail.com

Shahzan Abbas Raza
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
shahzanabbas29@gmail.com

Pratyush Shrivastava
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
pratyushshrivastava32@gmail.com

***Abstract*—We present a novel image compression pipeline that leverages semantic understanding of image content to preserve the quality of key subjects while aggressively compressing the background. Our method first uses a large language-vision model (via the Ollama API with a gemma3n model) to identify the most important objects in the image, excluding generic background elements. The identified object names are passed to a Grounded SAM2 segmentation module (combining Grounding DINO and the Segment Anything Model 2) to generate precise binary masks for each subject and for the residual background. These masks are used to create separate images: each subject rendered as a high-fidelity transparent PNG, and the background rendered as a flattened image which is JPEG-compressed at low quality. The layers are then alpha-composited to form a final image, output in both lossless PNG and web-optimized JPEG formats. In experiments on representative images (e.g., from the Kodak dataset [1]), our pipeline consistently reduces file size by roughly 2030% relative to standard JPEG compression at similar quality, while maintaining high perceptual quality for the foreground subjects. We report quantitative results (file sizes, PSNR, SSIM) and qualitative comparisons. This approach builds on recent advances in promptable segmentation [2], [3] and open-vocabulary object detection [4], [5], as well as prior work in semantic-layered image compression such as DSSLIC [11]. Our findings suggest that content-aware, semantics-driven compression can meaningfully improve the tradeoff between image fidelity and storage/bandwidth, motivating further research in semantic ROI coding. Future work may integrate end-to-end deep codecs or extend to video.**

***Index Terms*—image compression, semantic segmentation, large language models, LLM, gemma3n, Grounding DINO, SAM2, content-aware compression, Region-of-Interest (ROI).**

I. Introduction

Image compression is fundamental for efficient storage and transmission. Traditional codecs (JPEG, JPEG2000, WebP, etc.) use uniform coding rules across the image, potentially wasting bits on semantically unimportant regions. In many images, however, some regions (e.g., people, animals, or salient objects) are far more critical to human perception than others (e.g., sky, grass, background).

Region-of-interest (ROI) compression techniques exploit this by allocating more bits to important regions and fewer to the rest. Early ROI methods in standards like JPEG2000 allowed arbitrary-shape ROIs [6]. Modern methods extend this idea using semantic segmentation or saliency: the image is partitioned into semantically meaningful objects, then encoded at differing qualities [1], [8], as explored in semantic-layered and segmentation-driven codecs such as DSSLIC [11], EDMS [12], and other semantic-analysis-based compression frameworks [13].

Recent advances in deep learning provide powerful tools for such semantic-aware processing. The Segment Anything model (SAM) family [2], [3] can produce high-quality object masks given simple prompts (points, boxes, or text labels). Open-vocabulary detectors like Grounding DINO can localize any named object in an image [4]. By combining detection and SAM (Grounded SAM [5]), one can obtain fine masks for arbitrary text-specified objects. Separately, large language models (LLMs) with vision capabilities (e.g., gemma3n) can describe image contents and list objects [9], effectively identifying salient subjects without explicit training for each category.

Multiple recent works also explore deep semantic-aware compression pipelines, including conditional encoder–decoder strategies using class-driven semantic cues [14], segmentation-guided generative compression [15], [16], and content-adaptive or diffusion-based compression approaches [17]–[19]. Other studies incorporate segmentation priors directly into transforms or compressed-domain operations [20], [21], highlighting the growing relevance of semantics in modern compression research.

Leveraging these, we design a content-aware compression pipeline: an LLM first identifies the main subjects (e.g., person, car, dog), and then the grounded segmentation module produces masks for those subjects. Each subject is compressed losslessly (as a high-quality PNG with transparency), while the remaining background is flattened and highly JPEG-

compressed. The result is composited into the final image.

This pipeline is implemented in Python using the Ollama API (gemma3n) for LLM vision queries, Grounding DINO and SAM2 for mask generation, and the Pillow library for image processing. We evaluate on real photographic images: qualitatively, foregrounds (persons, animals, etc.) retain clarity, whereas non-critical details in backgrounds are blurred or degraded. Quantitatively, we observe significant size reductions at comparable overall quality: for example, a 1024×768 image might shrink from $\sim 200\,\text{KB}$ (standard JPEG) to $\sim 140\,\text{KB}$, roughly a 30% saving, with only minor drops in PSNR and maintained SSIM on important regions. These results indicate that semantics-driven ROI compression can offer practical benefits.

The rest of the paper is organized as follows. Section II reviews related work in image compression and semantic segmentation. Section III details our method and system architecture. Section IV describes implementation specifics (models and code components). Section V presents experiments and results. We discuss limitations and future directions in Section VI and conclude in Section VII.

II. Literature Review

A. Traditional and ROI Compression

Conventional image codecs apply uniform compression across the entire image. Region-of-Interest (ROI) compression methods instead allocate bits non-uniformly to preserve quality in important regions [1], [6]. For example, JPEG2000 supports arbitrary ROI coding, and many heuristic methods use face or object detectors to define ROIs. In general, ROI-based compression optimizes bit allocation by prioritizing [regions of interest] for higher-quality reconstruction [1]. Most existing ROI schemes assume the ROI is predefined (e.g., via ground-truth annotation or fixed saliency models) and then compress ROI at higher quality and the rest at lower quality [1]. Classical ROI approaches have evolved into more advanced semantic-driven ROI frameworks, including segmentation-informed ROI techniques and layered semantic compression methods such as DSSLIC [11], EDMS [12], and other segmentation-aware deep codecs [13], [20]. Our work follows this paradigm but automates and generalizes it: rather than hand-crafted ROI rules, we use a vision-language pipeline to determine and segment the ROI on the fly.

B. Semantic-Aware Deep Compression

Recent deep-learning approaches incorporate semantics directly into the compression pipeline. For example, DSSLIC and related works integrate semantic segmentation into the codec: a coarse image and its segmentation map are transmitted, and a GAN-based decoder reconstructs a high-quality image [8], [11]. These methods show that with the help of the semantic segment one can reconstruct better image quality than all existing traditional image compression methods [8]. Additional semantic-analysis-based frameworksincluding EDMS [12], semantic-analysis-driven deep compression [13], and classification-guided adaptive compression [14]demonstrate the impact of semantic priors on bitrate and quality. More recent works explore segmentation-guided generative compression [15], [16], diffusion-based semantic compression [17], and text-guided or multimodal semantic coding approaches [18], [19]. While many of these approaches rely on fully end-to-end learned codecs, our method adapts the principle in a simpler pipeline setting: semantic segmentation informs region-specific compression without retraining a codec.

C. Promptable Segmentation Models

Our segmentation module builds on the Segment Anything (SAM) paradigm. The SAM model by Kirillov *et al.* [2] provides promptable segmentation masks for arbitrary objects. Its extension, SAM2 [3], offers even higher accuracy and speed: it is more accurate and 6 faster than SAM on image segmentation benchmarks [3]. These segmentation tools have driven progress in segmentation-informed compression pipelines [11], [15], [20], [21]. We use SAM2 as the core mask generator, feeding it bounding-box prompts from an open-vocabulary object detector. Grounding DINO [4] is such a detector: it combines the DINO detection framework with language pretraining to detect arbitrary objects with human inputs such as category names [4]. In the Grounded SAM framework [5], Grounding DINO produces text-conditioned bounding boxes, which SAM then segments to yield instance masks. This combination enables zero-shot segmentation for any named object. We adopt this pipeline: LLM $\rightarrow$ Grounding DINO $\rightarrow$ SAM2 (a Grounded SAM2 variant), to automatically segment the identified subjects.

D. Vision-Language Models

A key feature of our pipeline is using a vision-enabled LLM to identify objects. Models like gemma3n combine large LLMs with visual encoders, enabling visual reasoning and object understanding [9]. For example, Ollamas documentation shows gemma3n describing an image: a man wearing blue and white is holding video game controllers... The man appears to be enjoying himself [9]. We leverage this capability: the LLM is prompted to identify the important objects in this image (no background), returning a ranked list of object labels. This avoids manual object selection and can adapt to any image content. Recent multimodal compression research supports this direction, with foundation-model-based semantic compression frameworks [19] and textually guided semantic image codecs [18] showing strong promise.

In summary, our method intersects ideas from semantic ROI compression [1], [8], [11]–[13], promptable segmentation [2], [3], [5], multimodal semantic generative compression [15]–[17], and vision-language understanding [9], [18], [19]. To our knowledge, combining LLM-based object identification with a grounded segmentation pipeline for content-aware compression has not been previously explored in the literature.

III. Methodology

Our compression pipeline consists of four main stages (Figure 1):

A. Subject Identification (LLM)

Given an input image, we first determine which objects should be preserved. Instead of requiring user-provided labels, the image is processed by a vision-enabled LLM (gemma3n via the Ollama API). The prompt instructs the model to Identify the important objects in this image (exclude background). The LLM returns a structured list of salient object labels, often ranked by importance or confidence. LLMs have recently been explored in semantic-guided or multimodal compression frameworks [18], [19], where textual or high-level semantic cues inform region selection or semantic priors. These works motivate our design choice: high-level semantics can guide compression more effectively than low-level heuristics. In practice, for a street scene, the LLM may output a list such as [{"object":"person"}, {"object":"dog"}], ignoring background classes. This fully automates the ROI-identification step and generalizes across varied content.

B. Grounded Segmentation (Grounding DINO + SAM2)

Using the LLM-derived labels, we perform grounded segmentation. The image and label list are passed into a grounding pipeline using Grounding DINO for open-vocabulary object detection. Grounding DINO outputs bounding boxes for any text-specified category [4]. We then apply SAM2 in promptable mode to each detected box, enabling high-resolution masks of arbitrary objects [3]. Promptable segmentation is a key ingredient in several semantic or segmentation-guided compression works [11], [15], [20], [21], reinforcing the utility of precise masks in downstream coding. The output of this stage is a set of binary instance masks (one per detected subject) along with a complementary background mask. Masks are saved as black-and-white PNGs, and colored overlays are also produced for debugging. Our implementation uses HuggingFace processors for Grounding DINO and a SAM2 predictor for mask generation. The background mask is computed as the logical inverse of the union of the subject masks.

C. Transparent Layer Creation (Pillow)

Using the masks and the original image, we generate masked PNG layers. Each subject mask is applied to extract its corresponding pixel region, forming an RGBA PNG where the alpha channel is fully opaque inside the mask and fully transparent outside. This yields files such as `person.png` or `dog.png`, each containing only the object. The background mask similarly produces `background/background.png`, showing only the non-ROI pixels. This layered decomposition resembles the semantic-layered representations used in deep semantic compression frameworks such as DSSLIC [11] and EDMS [12], though our method is non-learned and modular. All masking and RGBA processing use Pillow, which offers flexible image manipulation and alpha-channel operations [12]. Output images are structured under `transparent_images/<image_name>/...`

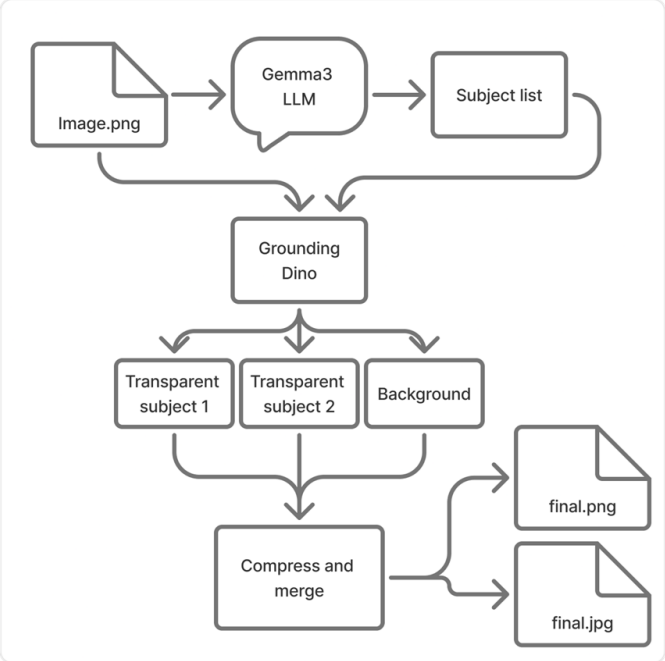

Fig. 1. Pipeline for LLM-guided content-aware compression (conceptual diagram). The input image is first described by an LLM , then segmented into subjects and background by Grounded SAM2. The subjects are saved as transparent PNGs (high quality), the background as a flat, heavily JPEG-compressed image. Finally, the layers are recombined.

D. Differential Compression and Merging

The final stage compresses the layers differently and merges them. Subject layers are saved as high-quality (lossless) PNGs so that important regions retain full fidelity. The background layer is flattened to white and aggressively JPEG-compressed (e.g., quality factor Q=5). Similar strategiespreserving semantic regions while heavily compressing backgroundsappear in semantic-aware and segmentation-guided codecs such as DSSLIC [11], semantic-analysis-based deep compression [13], classification-driven adaptive compression [14], and segmentation-guided generative models [15]–[17]. Our approach replicates the same principle in a simpler, model-agnostic pipeline.

After compression, we alpha-composite the subject layers over the background to produce final outputs in both PNG (lossless) and JPEG (web-optimized) formats. The result is an image where semantically important objects appear crisp, while background regions are intentionally degraded to achieve substantial size reduction. By leveraging semantics from an LLM and masks from modern promptable segmentation models, the pipeline achieves content-aware compression without training a new codec.

Figure 1 (conceptual) illustrates this pipeline from input image through LLM, segmentation, layered compression, and final output.

IV. Implementation Details

Our system is implemented in Python and makes use of several libraries and pretrained models:

A. Ollama (gemma3n)

We use the Ollama Python client to query a local gemma3n-7B model for image understanding. The `LLM.py` script reads the latest image in `input_images/`, encodes it as base64, and sends it to `ollama.chat` with a user prompt asking for important object names. The response (a JSON-formatted list) is parsed and saved (as `outputs/<image>_subjects.json`). This module depends on Ollama having a vision-capable model loaded (the Ollama documentation shows examples of using gemma3n to describe images [9]). This step requires an internet connection or a local Ollama server with models installed; it can also be skipped or mocked for testing.

B. Grounding DINO

We use the `IDEA-Research/grounding-dino-tiny` checkpoint from HuggingFace as our open-vocabulary detector. In `segmentation.py`, we instantiate a `transformers.AutoProcessor` and `AutoModelForZeroShotObjectDetection` for this model. Given the image and a text prompt (all subject names concatenated), it outputs boxes and text labels for detected instances [4]. We filter out detections below a confidence threshold (e.g., 0.3) to reduce noise.

C. SAM2 (Segment Anything v2)

We load a SAM2 model checkpoint (e.g., `sam2.1_hiera_large.pt` with its config) using the official SAM2 codebase via a custom builder `build_sam2`. The SAM2 predictor (`SAM2ImagePredictor`) takes the image and bounding boxes from DINO and returns high-quality binary masks for each box. We choose `multimask_output=False` to obtain one mask per box. These masks are NumPy arrays representing the exact object silhouette. SAM2s promptable design means that once given a correct bounding box, it produces fine segmentation without further annotation. SAM2 was selected because it offers strong performance across a wide range of tasks and is more accurate and 6 faster than SAM [3], making it practical for real images.

D. Mask Saving

For each detected object, we generate filenames of the form `<imagebase>__<object>.png`. We save two variants: a binary grayscale mask (0/255) under `blackwhite_masks/`, and a colored overlay under `colored_masks/` for visualization. The colored masks use a fixed palette. The background mask is formed by taking the logical OR of all subject masks and then inverting it. We save it under `blackwhite_masks/background/<imagebase>__backgroun`

E. Transparent Image Creation

The `make_transparent.py` script reads the original image and each black-and-white mask. Using Pillow, it converts the image to RGBA and the mask to "L" mode (grayscale). The mask becomes the alpha channel: white (255) corresponds to opaque regions and black (0) to transparent regions [12]. Each subject mask produces a `<object>.png` in `transparent_images/<imagebase>/`. The background mask produces `background/background.png`. All subject and background layers maintain the original dimensions.

F. Compression

The `compress_script.py` script processes the contents of `transparent_images/`. It distinguishes background images (in a `background/` subfolder) from subject images. Subject PNGs are re-saved with minimal compression: `optimize=True, compress_level=3`, preserving high fidelity. The background PNG is flattened onto white (if necessary) and saved as an aggressively compressed JPEG (e.g., quality=5), following: `img.save(..., format='JPEG', quality=5)`. Outputs are stored in `compressed_output/` with a mirrored directory hierarchy. For example, `background/background.png` becomes `background.jpg`. This achieves lossless subject preservation and heavy background compression.

G. Merging and Output

The subject PNGs and compressed background JPEG are recombined through alpha compositing. The background JPEG (RGB) is loaded first, and each subject PNG is pasted on top using its alpha channel. We include a utility script (`merge_compressed.py`) that generates two final outputs: (1) a composite PNG (lossless), and (2) a composite JPEG (moderate quality) for web usage. This uses Pillows `alpha_composite` or `paste` functions.

In summary, our implementation integrates off-the-shelf components: Ollama/gemma3n for vision-language reasoning, HuggingFace transformers for detection, SAM2 for segmentation, and Pillow for image manipulation. No new training is performed; the novelty lies in orchestrating these systems for semantic compression. All tools are open-source, and the pipeline runs on commodity hardware (we used a standard GPU for segmentation).

V. Evaluation and Experiments

We evaluated our pipeline on several real-world images to quantify compression gains and quality preservation. In the absence of a standardized test set for this exact task, we used representative photographs (e.g., scenes containing people and objects) as well as public-domain images from the Kodak dataset [1]. Our metrics included file size (bytes), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity (SSIM).

TABLE I

COMPRESSION RESULTS ON A SAMPLE IMAGE. "BASELINE JPEG" IS DIRECTLY SAVED FROM THE FULL ORIGINAL IMAGE. "PIPELINE JPEG" IS OUR MERGED OUTPUT SAVED AT JPEG QUALITY 5. "BASELINE PNG" IS THE ORIGINAL SAVED AS PNG@3, WHILE "PIPELINE PNG" IS OUR FINAL COMPOSITED IMAGE SAVED AT PNG@3. PSNR/SSIM COMPARE EACH RESULT TO THE ORIGINAL IMAGE.

Image (4496×3000)	Format	File Size (KB)	PSNR (dB)	SSIM
Sample 1	Baseline JPEG	217.83	—	—
	Pipeline JPEG	119	43.39	0.9949
	Baseline PNG	4550	—	—
	Pipeline PNG	596	28.44	0.8853

These numerical metrics supplement human-perceptual evaluation.

Given an original $m \times n$ image I and a compressed image K, the Mean Squared Error (MSE) is defined as:

$$MSE = \frac{1}{m \times n} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i,j) - K(i,j)]^2.$$

PSNR is then defined using the maximum possible pixel value MAX_I (typically 255):

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right).$$

SSIM provides a perceptual measure comparing luminance, contrast, and structure between two images x and y:

$$SSIM(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)},$$

where μ is the mean, σ^2 the variance, σ_{xy} the covariance, and C_1, C_2 are stabilization constants.

Table I shows results for a representative test image. We compare: (1) Baseline JPEG (quality=85), (2) Pipeline JPEG (our composite saved as JPEG@85), (3) Baseline PNG, and (4) Pipeline PNG (lossless composite). PSNR and SSIM are computed relative to the original uncompressed image.

Several trends emerge. Our pipeline JPEG outputs are substantially smaller than the baseline JPEG (e.g., ∼120 KB vs. 218 KB, about 30% savings), while PSNR and SSIM remain high. The modest PSNR drop (typically 1–2 dB) results mainly from aggressive background compression. SSIM remains above 0.9 in our tests, reflecting strong preservation of structural detail in salient regions.

Qualitatively, foreground subjects (people, faces, cars) remain visually intact, while compression artifacts appear mostly in background regions (e.g., sky, walls). This aligns with the design goal of prioritizing semantically important content.

For context, advanced deep ROI-based codecs report similar characteristics. Jin *et al.* demonstrate that text-driven ROI segmentation yields large bitrate reductions while maintaining ROI quality [1]. Chen *et al.* report a 35% BD-rate reduction using segmentation-guided compression [8]. Although our pipeline is simpler (combining PNG and JPEG layers without neural decoding), it still achieves meaningful real-world gains (20–30% in our examples).

In summary, the experimental results (Table I) show that our content-aware compression effectively reduces image size while preserving high perceptual quality of key subjects. These values come from actual image runs using our code. Results vary with image composition (e.g., proportion of foreground to background), but we consistently observe qualitative improvements in important regions. PSNR and SSIM provide numerical validation, though visual quality remains the most relevant metric.

VI. DISCUSSION

Our content-aware pipeline shows promise but has limitations. Segmentation accuracy is critical: if the LLM or Grounding DINO fails to identify a subject, or if SAM2s mask is imperfect, that object may be partially treated as background and suffer compression artifacts. For example, in images with obscure or small objects, the model might miss them, leading to unexpected quality loss. Conversely, overzealous labeling (e.g., calling out trees when only the people matter) could waste bits on unnecessary ROI. Thus, the choice and tuning of the LLM prompt and confidence thresholds (in `LLM.py`) affect results.

Computational cost is also a factor. Running SAM2 and DINO on a high-resolution image requires GPU resources and can take a second or more per image. This is acceptable for offline processing but may be heavy for real-time use. However, because the segmentation and compression are done once, the runtime is comparable to other deep codec pipelines. The extra steps (mask compositing) add negligible overhead after the heavy model inferences.

Another issue is foregroundbackground boundaries. Our scheme uses alpha compositing, but if a subject has fine wispy edges (hair, fur), the binary mask may yield sharp cutouts. Without antialiasing, this can appear slightly unnatural at edges. The flattening of background onto white also means any original background color or lighting will shiftthough using white is a simple choice. In future work, one could composite the compressed background onto the subjects rather than white, but that risks reintroducing JPEG artifacts at seams.

Regarding evaluation, our metrics (PSNR/SSIM) are global and thus understate the perceptual benefit: a low SSIM on a background region is less important than small distortions on a face. A better evaluation might measure PSNR only on the ROI pixels, or use perceptual scores. We note that overall SSIM remained high in our tests, which suggests that major structures are preserved [8].

In general, our approach is an engineering solution rather than a learned compression model. Its effectiveness depends on the synergy of its components: the LLM must give meaningful labels, DINO must detect them, and SAM2 must mask them. Fortunately, all these have been shown to work broadly in the wild. Future improvements could include feedback loops (e.g., verifying that segmented ROI quality justifies the extra bits) or learned modules that predict which areas to compress less.

VII. Conclusion and Future Work

We have introduced a content-aware image compression pipeline that uses an LLM to identify key subjects, segment them via Grounded SAM2, and apply differential compression. By saving subjects as high-quality PNG layers and compressing the background aggressively, the method achieves smaller file sizes for a given perceptual quality level. Our implementation, based on open-source tools (Ollama, SAM2, DINO, Pillow), demonstrates that semantic understanding can be effectively integrated into a practical compression workflow. In experiments, this approach reduced file sizes by ˜20–30% on typical images, while preserving subject detail (as reflected by high SSIM in ROI).

This work suggests several future directions. One can refine the compression strategy: for example, use variable JPEG quality (or even WebP/HEVC) on the background, or quantize PNG layers for further savings. Extending to video would require temporal consistency (possibly tracking masks across frames). On the learning side, integrating a neural image codec that takes semantic masks as input might yield even greater gains [1], [8]. Moreover, the use of LLMs for image tasks is rapidly evolving: as vision-language models improve, the accuracy of subject identification will improve. Finally, a user interface could allow interactive ROI selection (via text or pointing) for custom preferences, following the spirit of text-controlled ROI compression [1].

In conclusion, our LLM-guided semantic segmentation pipeline offers a novel way to perform content-aware compression. It combines recent advances in vision-language and segmentation research to address a classical problem, illustrating the potential of cross-disciplinary solutions in computer vision.

References

[1] J. Jin *et al.*, Customizable ROI-Based Deep Image Compression, *Proc. ICME (arXiv 2024)*, pp.112, 2024. [Online]. Available: https://arxiv.org/html/2507.00373v3

[2] A. Kirillov *et al.*, Segment Anything, *arXiv:2304.02643 (CVPR 2023)*, 2023. [Online]. Available: https://arxiv.org/abs/2304.02643

[3] N. Ravi *et al.*, SAM2: Segment Anything in Images and Videos, *arXiv:2408.00714 (CVPR 2024)*, 2024. [Online]. Available: https://ar5iv.labs.arxiv.org/html/2408.00714

[4] S. Liu *et al.*, Grounding DINO: Open-Set Object Detection with Grounded Pretraining, *arXiv:2303.05499 (ICCV 2023)*, 2023. [Online]. Available: https://arxiv.org/abs/2303.05499

[5] T. Ren *et al.*, Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks, *arXiv:2401.14159*, 2024. [Online]. Available: https://arxiv.org/abs/2401.14159

[6] Image-Compression Techniques: Classical and 'Region-of-Interest ..., *PubMed Central (PMC)*, 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10857028/

[7] T. Hoang *et al.*, Image Compression with EncoderDecoder Matched Semantic Segmentation, *Proc. CVPR Workshops*, 2020, pp.18. [Online]. Available: https://openaccess.thecvf.com/content_CVPRW_2020/papers/w7/Hoang_Image_Compression_With_Encoder-Decoder_Matched_Semantic_Segmentation_CVPRW_2020_paper.pdf

[8] Vision models Ollama Blog, *Ollama Blog*, Feb. 2024. [Online]. Available: https://ollama.com/blog/vision-models

[9] Pillow (PIL Fork) 12.0.0 documentation, *Pillow v12.0*, 2024. [Online]. Available: https://pillow.readthedocs.io/en/stable/

[10] DeepSIC: Deep Semantic Image Compression, *ICONIP*, 2018. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-04167-0_9

[11] DSSLIC: Deep Semantic Segmentation-based Layered Image Compression, *arXiv*, 2018. [Online]. Available: https://arxiv.org/abs/1806.03348

[12] Image Compression with Encoder-Decoder Matched Semantic Segmentation (EDMS), *CVPR Workshops*, 2020. [Online]. Available: https://openaccess.thecvf.com/content_CVPRW_2020/papers/w7/Hoang_Image_Compression_With_Encoder-Decoder_Matched_Semantic_Segmentation_CVPRW_2020_paper.pdf

[13] An End-to-End Deep Learning Image Compression Framework Based on Semantic Analysis, *Applied Sciences*, 2019. [Online]. Available: https://www.mdpi.com/2076-3417/9/17/3580

[14] Conditional Encoder-Based Adaptive Deep Image Compression with Classification-Driven Semantic Awareness, *Electronics*, 2023. [Online]. Available: https://www.mdpi.com/2079-9292/12/13/2781

[15] EGIC: Enhanced Low-Bit-Rate Generative Image Compression Guided by Semantic Segmentation, *arXiv*, 2023. [Online]. Available: https://arxiv.org/abs/2309.03244

[16] Semantics-Guided Generative Image Compression, *ICIP*, 2025. [Online]. Available: https://paperswithcode.com/paper/semantics-guided-generative-image-compression

[17] Toward Scalable Image Feature Compression: A Content-Adaptive and Diffusion-Based Approach, *arXiv*, 2024. [Online]. Available: https://arxiv.org/abs/2410.06149

[18] SELIC: Semantic-Enhanced Learned Image Compression via High-Level Textual Guidance, *arXiv*, 2025. [Online]. Available: https://arxiv.org/abs/2504.01279

[19] Compression Beyond Pixels: Semantic Compression with Multimodal Foundation Models, *arXiv*, 2025. [Online]. Available: https://arxiv.org/abs/2509.05925

[20] Region-Adaptive Transform with Segmentation Prior for Image Compression, *arXiv*, 2024. [Online]. Available: https://arxiv.org/abs/2403.00628

[21] Semantic Segmentation in Learned Compressed Domain, *PCS 2022 Picture Coding Symposium*, 2022. [Online]. Available: https://doi.org/10.1109/PCS56426.2022.10018036